

AcademiX: Entrega 2

Diseño preliminar, plan de desarrollo, riesgos y walking skeleton
IIC3143 Desarrollo de Software · Equipo Lock In · Octubre 2026

Resumen ejecutivo

AcademiX es un LMS multi-tenant universitario enfocado en el flujo académico crítico: cursos, secciones, módulos, material, quizzes, calificaciones, publicación explícita de notas y auditoría. Esta entrega consolida el diseño preliminar y reduce complejidad respecto de Entrega 1: se abandona la base de datos por tenant y el Tenant Registry, reemplazándolos por PostgreSQL compartida, *Institution* como tenant lógico e *institution_id* como frontera de datos.

La arquitectura objetivo conserva robustez operacional: Railway como plataforma de despliegue, réplicas de frontend/backend, PgBouncer, PostgreSQL HA y Railway Bucket S3-compatible para material académico. El walking skeleton demuestra la ruta de integración: frontend, backend, Docker, CI, Railway y release por tag.

Trazabilidad contra pauta

Área pauta	Estado	Evidencia
Casos de uso actualizados	Cubierto	03-use-cases-and-requirements.md: CU-01 a CU-16 con actores, stakeholders, pre/postcondiciones, triggers, flujos y RNF.
Documento de diseño preliminar	Cubierto	Arquitectura, dominio, datos, walking skeleton y decisiones documentadas.
Arquitectura y demo/POC	Cubierto	04-architecture.md, 09-walking-skeleton-plan.md, 11-cicd-evidence.
Modelo de dominio	Cubierto	05-domain-model.md y 05-domain-model-uml.md.
Modelo de datos	Cubierto	06-data-model.md y catálogo XLSX/CSV en 12-data-catalog.md.
Plan de desarrollo	Cubierto	08-work-plan.md: siete semanas desde el lunes 5 de octubre de 2026.
Riesgos actualizados	Cubierto	07-risks.md: probabilidad, impacto, responsable, mitigación y contingencia.
PPT, informe y anexos	Cerrado	Informe, presentación Beamer, guía oral y anexos generados desde docs/deliveries/2.

Alcance funcional

El MVP considera dos flujos principales: docente y estudiante. El docente administra cursos, secciones, roles por Enrollment, módulos, material, quizzes y libro de notas. El estudiante elige institución, accede a cursos autorizados, consulta material publicado, responde quizzes y revisa notas publicadas.

Quedan fuera del MVP: administrador institucional, RBAC completo, rol coordinador, IA, chat, calendario, workers, Redis, microservicios y entregas manuales. La decisión evita que el diseño académico crezca más rápido que la capacidad de implementación.

Flujo	Casos de uso
Acceso	CU-01 elegir institución; CU-02 entrar al contexto institucional.
Cursos	CU-03 ver cursos y secciones; CU-04 gestionar secciones y roles.
Material	CU-05 publicar material; CU-06 consultar material; CU-14 organizar módulos.
Quizzes	CU-07 crear/publicar quiz; CU-08 responder; CU-09 enviar y calificar; CU-15 cancelar intento.
Notas	CU-10 configurar libro; CU-11 publicar; CU-12 ver notas; CU-16 revisar gradebook.
Auditoría	CU-13 consultar historial académico.

Arquitectura preliminar

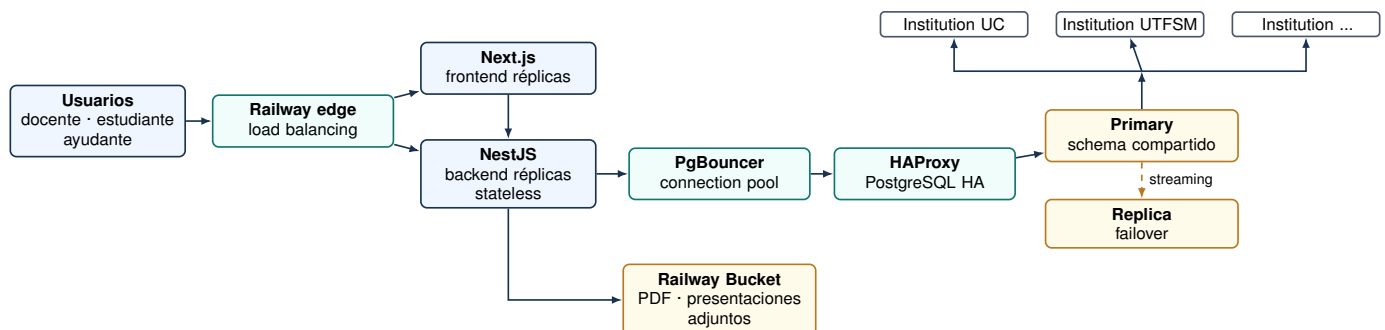


Figura 1. Arquitectura objetivo: Railway, réplicas de aplicación, pooling, PostgreSQL HA, object storage y separación lógica por *institution_id*.

La decisión central separa dos preocupaciones. La continuidad operacional se aborda con réplicas, healthchecks, PgBouncer,

PostgreSQL HA, backups y PITR. El aislamiento de datos se aborda con Institution, autorización contextual, queries scopeadas, institution_id, FK compuestas, constraints y auditoría.

Falla		Respuesta esperada	Límite
Réplica	fron-	Railway balancea hacia réplicas sanas.	Requiere más de una réplica.
tend/backend			
Backend completo		Restart policy y rollback si corresponde.	Un bug global requiere corrección.
Primary PostgreSQL		PostgreSQL HA promueve una réplica y HAProxy redirige.	Conexiones en vuelo deben reintentar.
Mala	mi-	Backups y PITR.	HA no corrige errores lógicos replicados.
gración/corrupción			
Object storage		Core sin archivos puede seguir.	Cargas/descargas fallan temporalmente.
Proveedor/región		Riesgo aceptado del MVP.	No se implementa multi-cloud.

Modelo de dominio

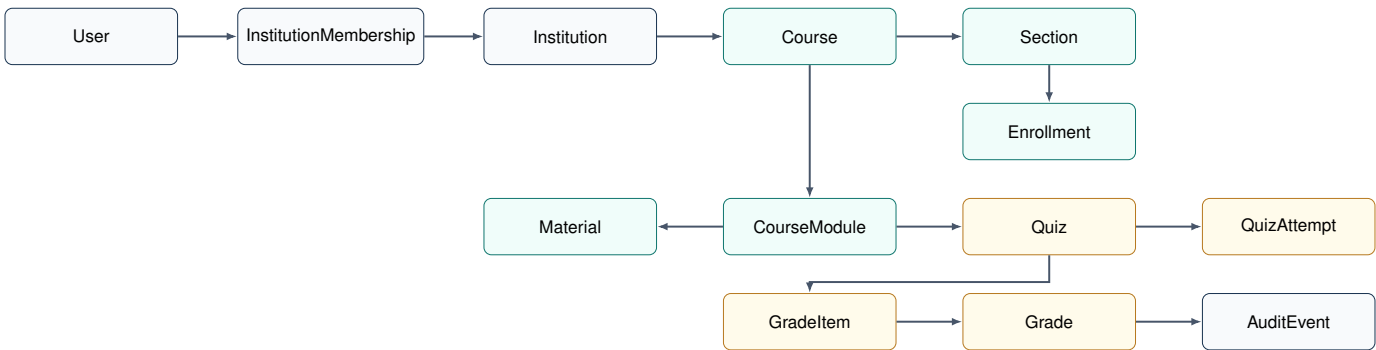


Figura 2. Dominio resumido del MVP.

Enrollment es el pivote académico: indica quién pertenece a qué curso y sección, en qué institución y con qué rol (teacher, assistant, student). No se modela coordinador ni RBAC completo. GradeItem representa una columna del libro de notas; Grade representa la nota concreta de un estudiante para ese ítem.

Modelo de datos

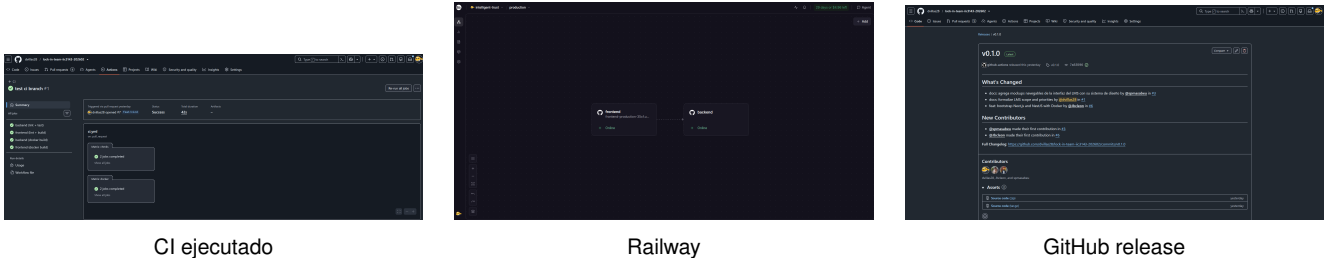
El modelo usa una sola PostgreSQL compartida y un schema evolutivo. Las tablas globales son institutions, users e institution_memberships. Las entidades académicas son tenant-owned y llevan institution_id.

Categoría	Tabla	Descripción
Global	institutions	Tenant lógico: slug, nombre, estado y ciclo de vida.
Global	users	Identidad global reutilizable entre instituciones.
Acceso	institution_memberships	Habilita acceso de usuario a institución.
Curso	courses, sections	Curso semestral y separación lógica por sección.
Curso	enrollments	Une usuario, institución, curso, sección y rol académico.
Contenido	course_modules, materials	Organización del material común visible para secciones del curso.
Evaluación	quizzes, quiz_attempts	Quiz con preguntas JSONB e intentos estudiantiles.
Notas	grade_items, grades	Columnas del libro y notas concretas por estudiante.
Auditoría	audit_events	Historial inmutable de cambios sensibles.

El catálogo de datos se adjunta como XLSX/CSV e incluye tablas, columnas, tipos, PK/FK, nulabilidad y descripciones.

Walking skeleton y CI/CD

El walking skeleton valida riesgos técnicos de integración antes de construir el flujo académico completo. La evidencia incluye frontend Next.js, backend NestJS, GET /health, Dockerfiles, GitHub Actions, configuraciones Railway y release por tag.



Plan de desarrollo

El plan parte el lunes 5 de octubre de 2026 y cubre siete semanas. Daniel Fierro aborda el bot PR reviewer; Sebastián Palma backend e infraestructura; Julián Contreras y Matías frontend; Daniel Villaseñor CI/CD, Scrum Master y gestión.

It.	Fechas	Foco	Responsables
1	2026-10-05 2026-10-11	a Plan ejecutable, CI/CD, contrato y presentación.	Villaseñor, Palma
2	2026-10-12 2026-10-18	a PostgreSQL compartida e identidad institucional.	Palma
3	2026-10-19 2026-10-25	a Contexto institucional y aislamiento.	Palma, Fierro
4	2026-10-26 2026-11-01	a Cursos, secciones, roles y shell frontend.	Palma, Contreras, Matías
5	2026-11-02 2026-11-08	a Módulos, material y quizzes.	Contreras, Matías, Palma
6	2026-11-09 2026-11-15	a Notas, publicación, auditoría y PR reviewer.	Palma, Fierro, Frontend
7	2026-11-16 2026-11-22	a Hardening, release, documentación y ensayo.	Todo el equipo

Riesgos actualizados

La planilla de riesgos concentra riesgos técnicos y de gestión con responsable, probabilidad, impacto, mitigación y contingencia. Los riesgos más importantes son fuga entre Institutions, queries sin scope institucional, sobrealcance, auditoría incompleta, CI/CD que no refleje el estado real y presentación final demasiado densa.

ID	Riesgo	Responsable	Mitigación
R-01/R-02	Fuga o query sin scope institucional	Sebastián Palma	Contexto centralizado, FK institution-aware y tests UC/UTFSM.
R-09/R-10	Railway o CI/CD mal configurado	Daniel Villaseñor	Healthchecks, Docker reproducible, pipelines y release por tag.
R-12	Sobrealcance funcional	Daniel Villaseñor	Recorte al flujo curso-material-quiz-notas.
R-13	Auditoría incompleta	Sebastián Palma	AuditEvent en misma transacción y tests de cambios sensibles.
R-15	Frontend permite acciones fuera de rol	Julián Contreras, Matías	UI derivada de permisos; backend conserva autorización final.
R-16	Bot PR reviewer genera ruido	Daniel Fierro	Activación gradual y revisión humana obligatoria.

Conclusión

La Entrega 2 deja una base estable para construir: casos de uso trazados, arquitectura defendible, modelo de dominio, modelo de datos, plan de trabajo, riesgos y walking skeleton. La simplificación frente a Entrega 1 no elimina garantías críticas; elimina complejidad prematura y conserva las defensas que importan: aislamiento tenant-aware, autorización contextual, auditoría, CI/CD reproducible y operación con HA/pooling/storage.

Anexos declarados

Los anexos de la carpeta docs/deliveries/2 incluyen catálogo de datos XLSX, CSV de tablas/columnas/constraints, OpenAPI, capturas CI/CD, evidencia Railway, riesgos completos y plan de desarrollo detallado.